AWS Cloud Project Documentation

QuickLoan

Scalable Web Application Deployment on Amazon Web Services

A complete cloud infrastructure project - VPC EC2 RDS ALB Auto Scaling S3

Prepared by: Sujal Phadale

Guided by: Mr. Lekharaj (Trainer)

Project Specifications

TECHNOLOGY STACK: PHP, Apache/Nginx, MySQL, AWS	CLOUD PLATFORM: Amazon Web Services (AWS)
REGION USED: Mumbai Region	DOCUMENT TYPE: Step-by-Step Technical Documentation

Table of Contents

1. Project Overview & Architecture

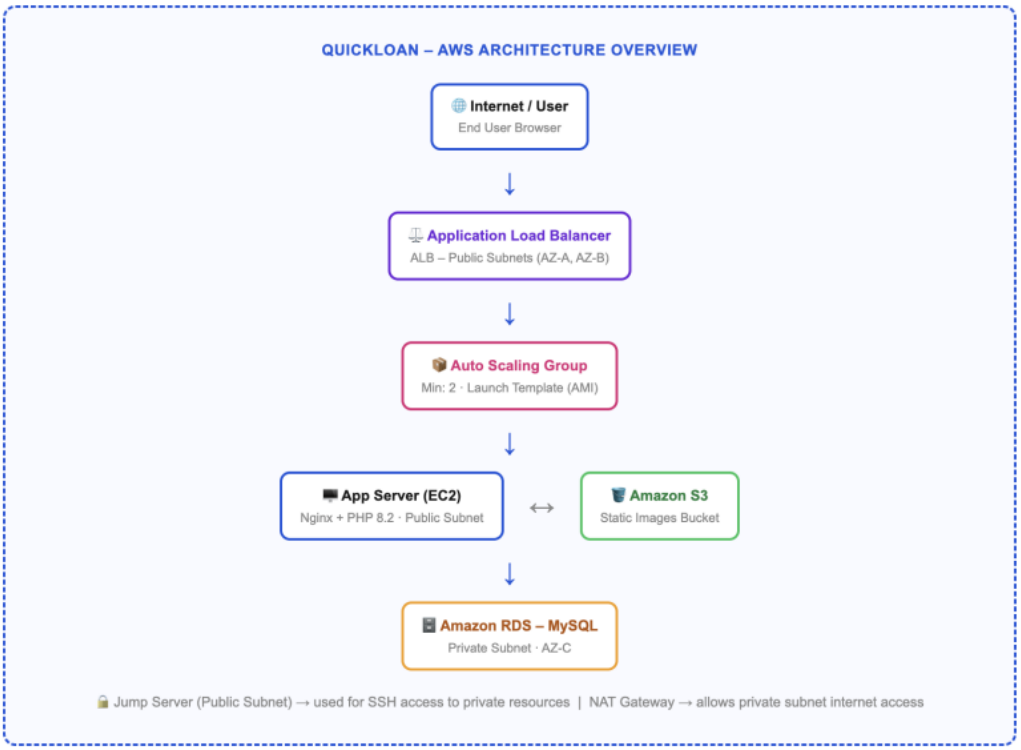
QuickLoan is a cloud-hosted loan application web platform built with a PHP backend and HTML/CSS/JavaScript frontend. The project demonstrates a production-grade AWS infrastructure setup encompassing networking, compute, storage, database, auto scaling, and load balancing - following industry best practices for high availability and fault tolerance.

The application was deployed inside a custom Virtual Private Cloud (VPC) with properly segmented public and private subnets across multiple Availability Zones. Traffic flows through an Application Load Balancer (ALB) to an Auto Scaling Group of EC2 instances, which communicate with a managed RDS MySQL database in a private subnet. Static assets (images) are served from Amazon S3.

2. Technology Stack

LAYER	TECHNOLOGY/SERVICE	PURPOSE
Frontend	HTML, CSS, JavaScript	User interface of the QuickLoan web app
Backend	PHP 8.2, php-fpm, php-mysqlnd	Server-side application logic
Web Server	Nginx	HTTP request handling and routing
Database	Amazon RDS-MySQL (MariaDB client)	Persistent storage of loan applications
Object Storage	Amazon S3	Hosting static image assets
Networking	VPC, Subnets, IGW, NAT Gateway, Route Tables	Isolated, secure cloud network
Compute	Amazon EC2 (Amazon Linux 2023)	Application, jump, and database servers
Scaling	Auto Scaling Group + Launch Template (AMI)	Horizontal scaling based on demand
Load Balancing	Application Load Balancer (ALB)	Traffic distribution across EC2 instances
DNS/Domain	No-IP (quickloan.hopto.org)	Free dynamic DNS for application access
File Transfer	WinSCP	Uploading application files to EC2

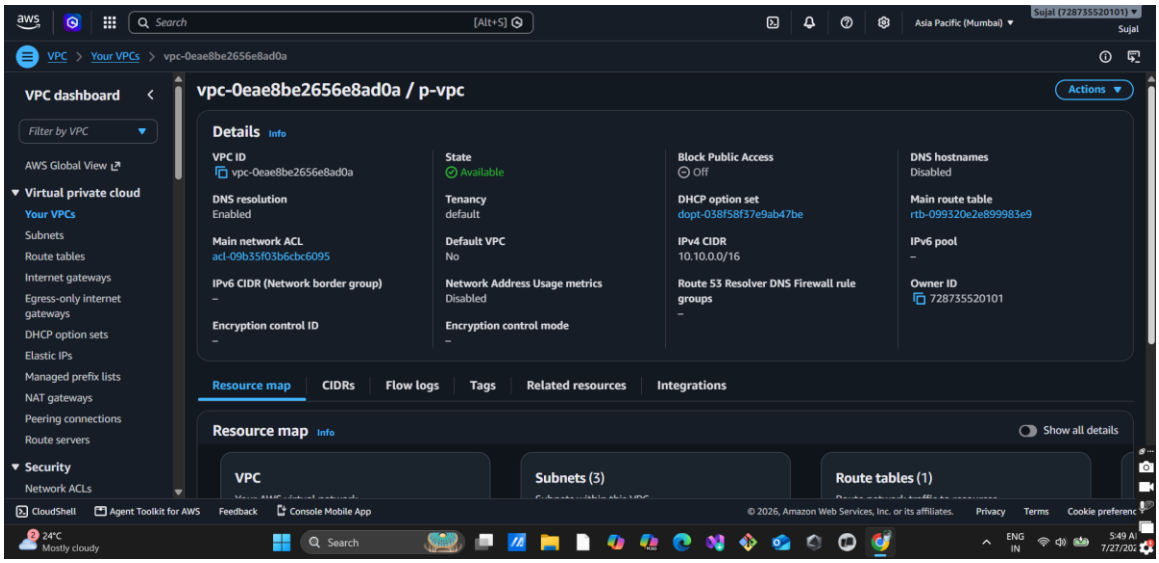
3. Architecture Flow Diagram



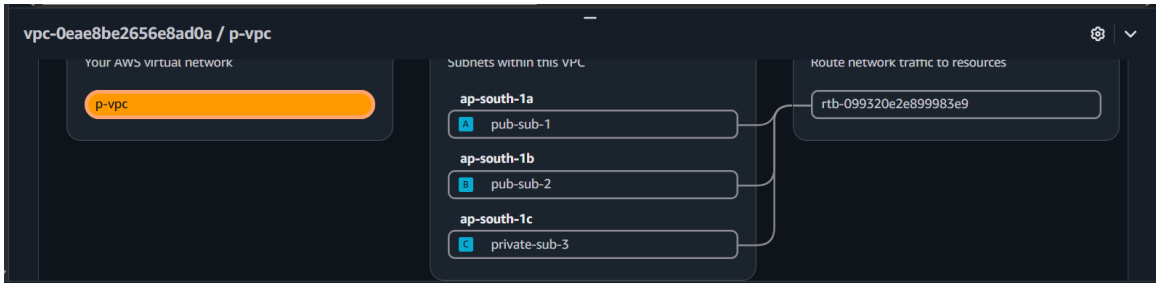
The flow begins with a user request hitting the ALB, which forwards it to one of the healthy EC2 instances managed by the Auto Scaling Group. Each EC2 instance runs Nginx + PHP and serves the QuickLoan frontend. Static images are loaded from S3. Application data (loan submissions) is stored in the RDS MySQL database in the private subnet. The Jump Server acts as the secure SSH entry point for all administrative access.

4. Step 1 - VPC & Networking Setup

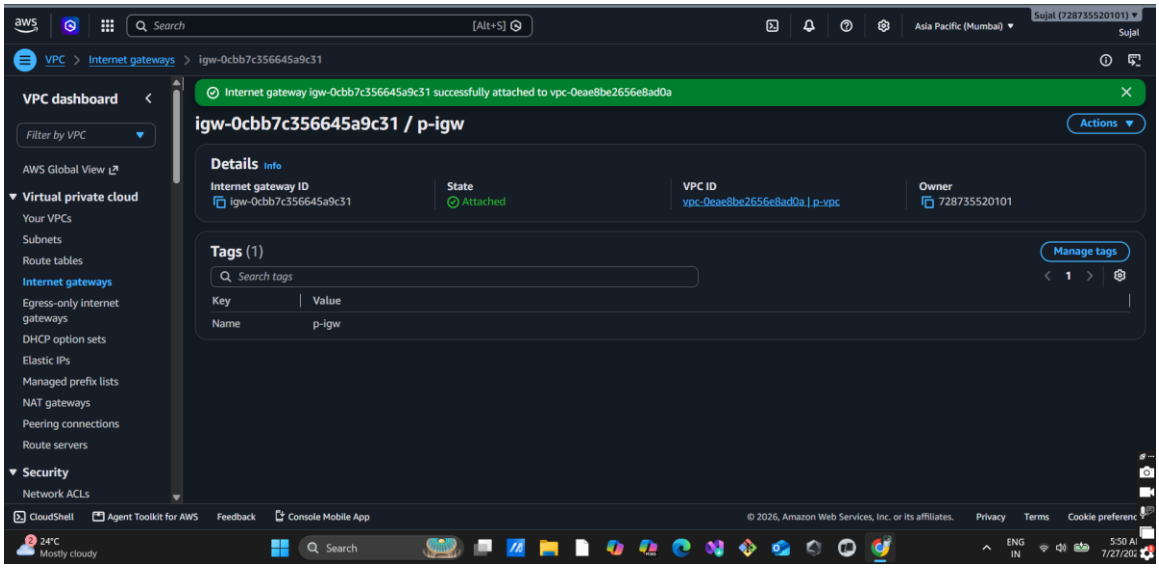
1.1 Create VPC: A custom Virtual Private Cloud (VPC) was created to define an isolated network environment.



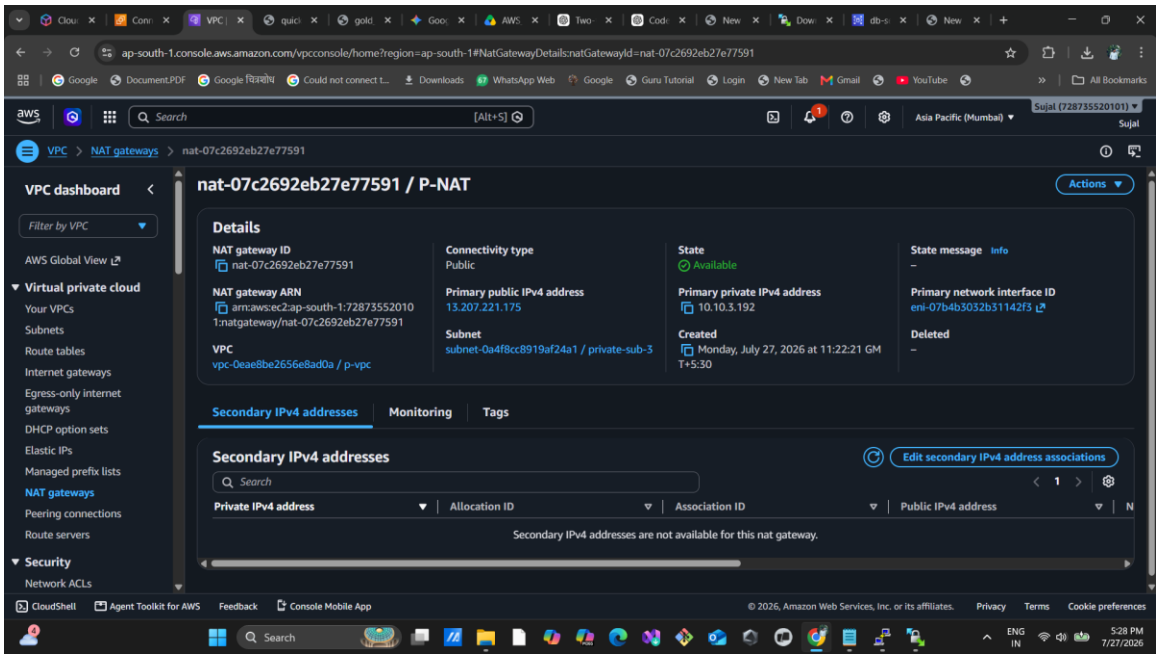
1.2 Create 3 Subnets: Public Subnet 1 (AZ A), Public Subnet 2 (AZ B), and Private Subnet 3 (AZ C).



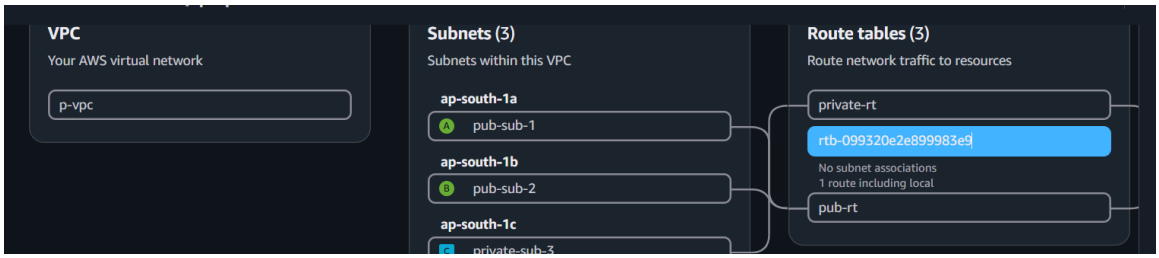
1.3 Internet Gateway (IGW): Attached to VPC to enable resources in public subnets to communicate with the internet.



1.4 NAT Gateway: Deployed in Public Subnet 1 to allow private instances outbound internet access.

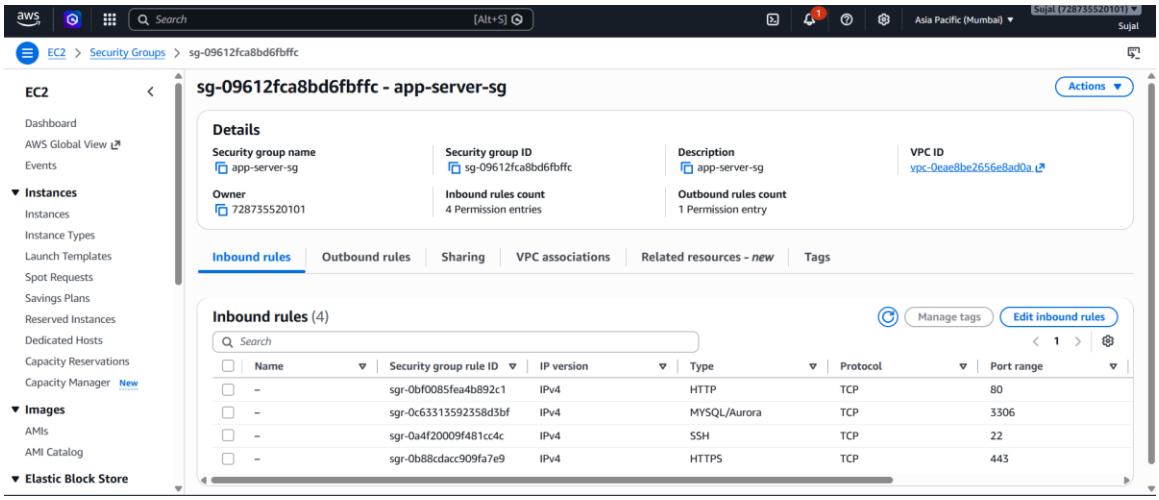


1.5 Route Tables: Configured Public RT (to IGW) and Private RT (to NAT Gateway).

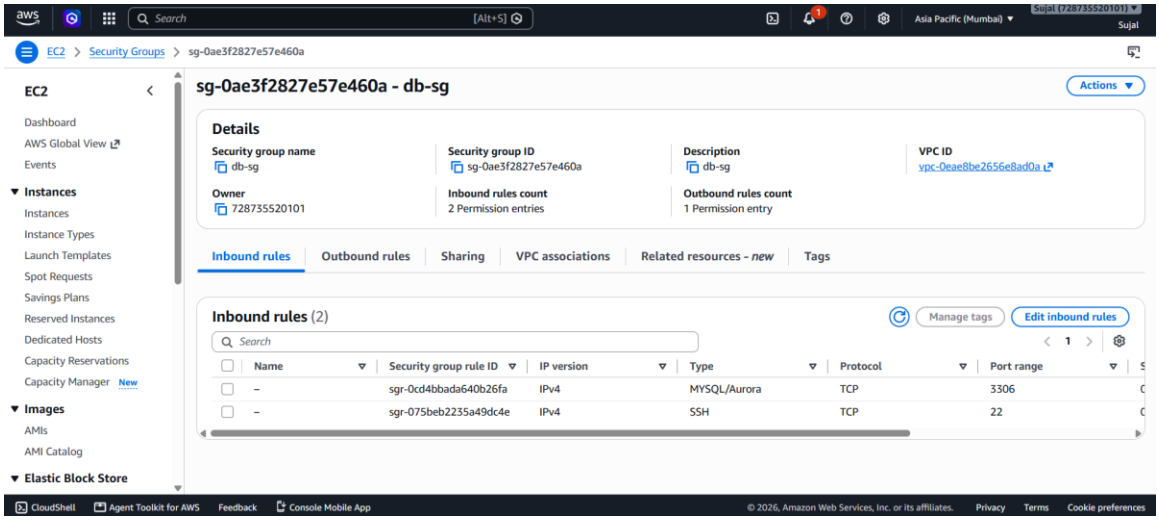


5. Step 2 - Security Groups Configuration

SG-1 App & Jump Server: Inbound Port 22 (SSH), Port 80 (HTTP).

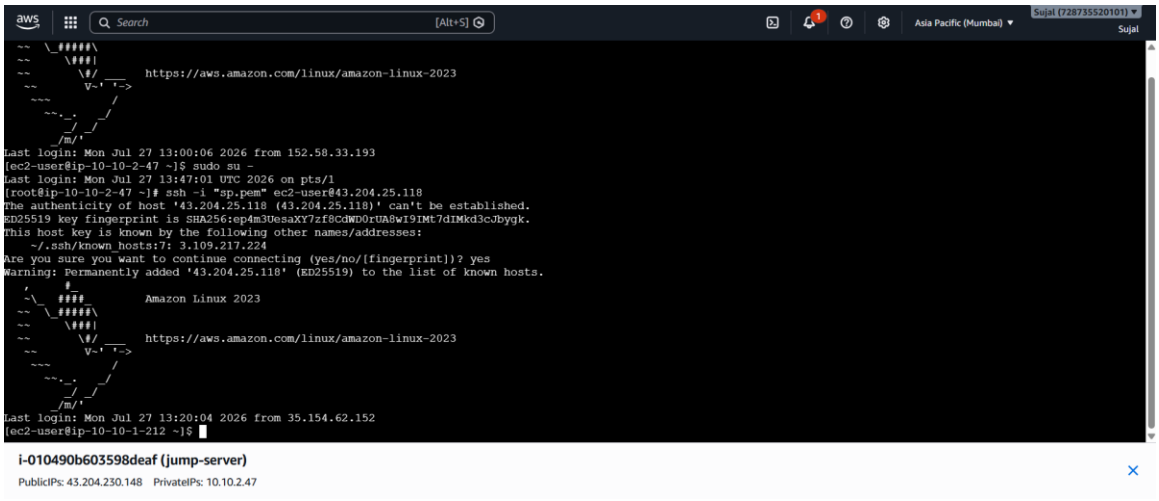


SG-2 Database Server: Inbound Port 22, Port 3306 (MySQL) restricted to App Server SG.



6. Step 3 - EC2 Instance Launch

Three EC2 instances launched on Amazon Linux 2023: jump-server (Public), app-server (Public), database-server (Private). All server access was performed using the SSH jump pattern via the Bastion Host.



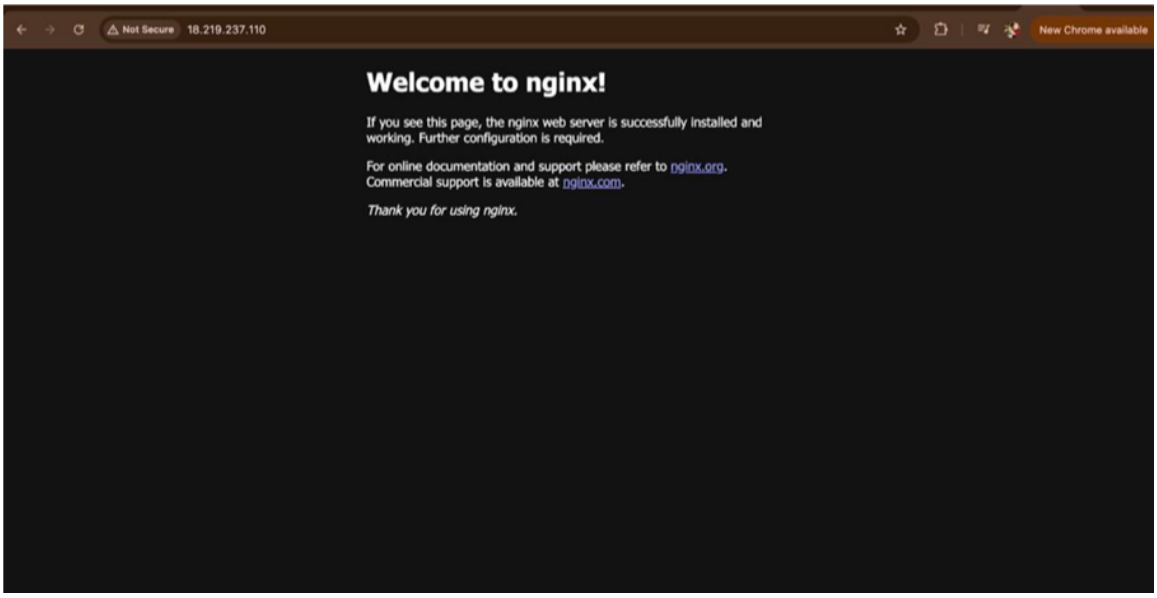
7. Step 4 - Application Deployment (Nginx + PHP)

Installed Nginx and PHP 8.2. Uploaded application files via WinSCP and moved to Nginx web root directory (/usr/share/nginx/html). Set appropriate ownership and permissions.

```
yum update -y

yum install -y nginx

systemctl start nginx

chkconfig nginx on

# Verify: Open browser → http://<public-ip>:80 — should show Nginx default page
```

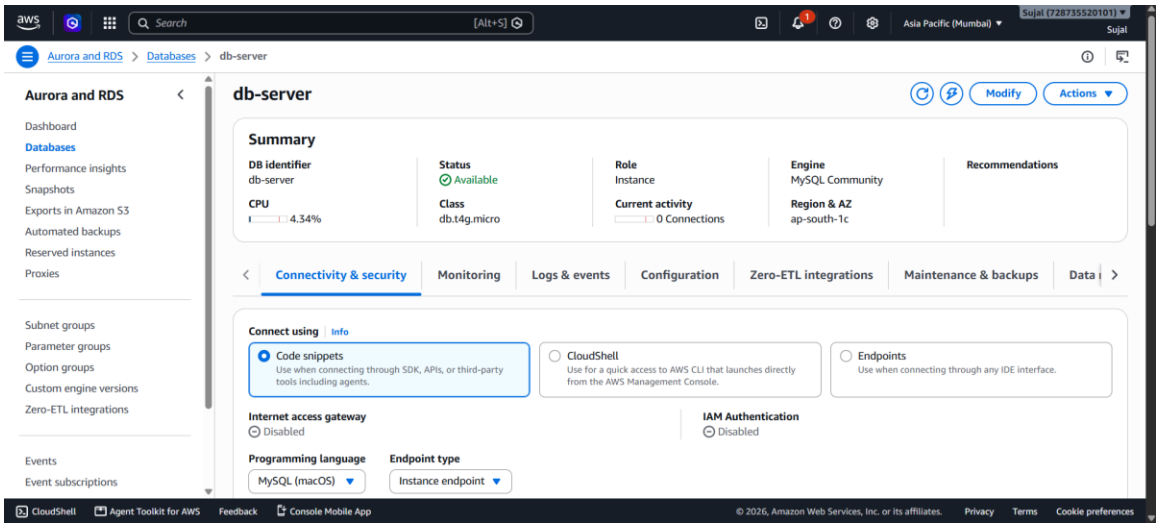
8. Step 5 - S3 Bucket for Static Assets

Created S3 bucket in us-east-2 to store static image assets. Updated application source files to reference the new S3 bucket URL.



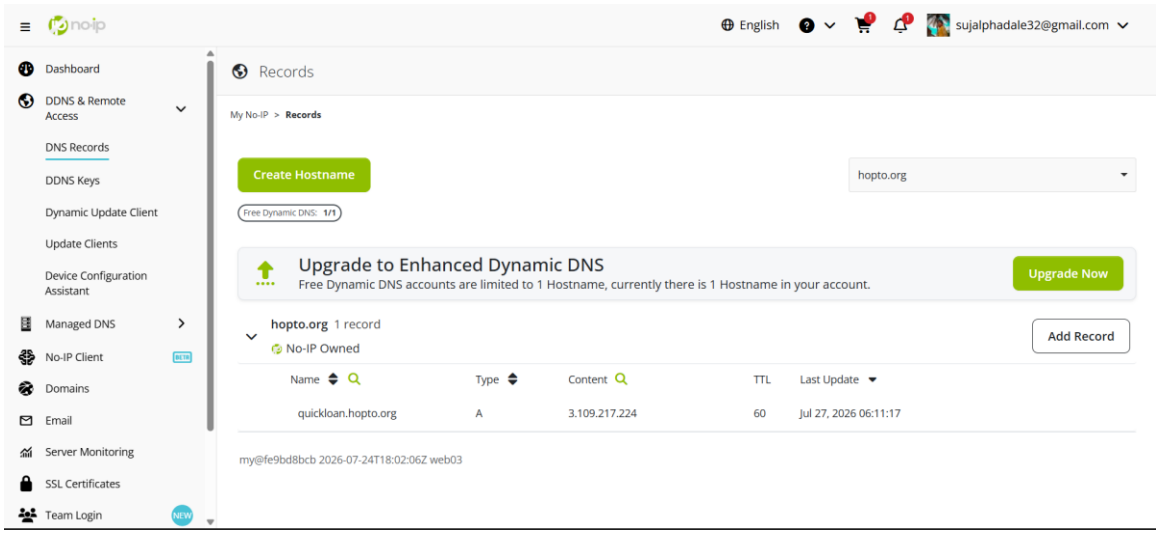
9. Step 6 - RDS MySQL Database Setup

Provisioned RDS MySQL instance in Private Subnet. Updated DB connection configuration on App Server. Created quickloan_db and applications table via Jump Server.



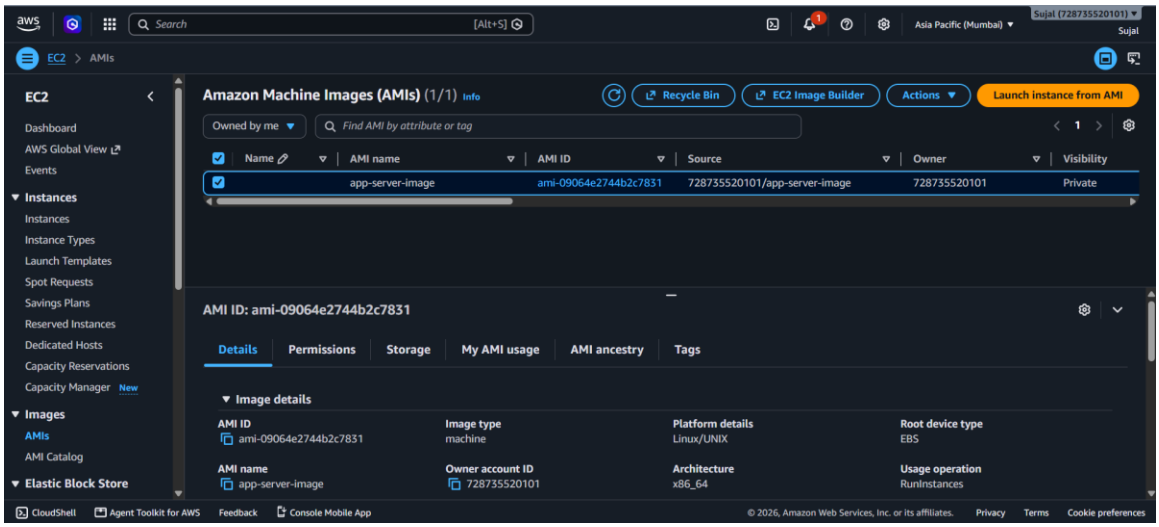
10. Step 7 - Domain Configuration (No-IP)

Registered free dynamic DNS hostname at noip.com. Updated Nginx virtual host configuration to use the domain name.



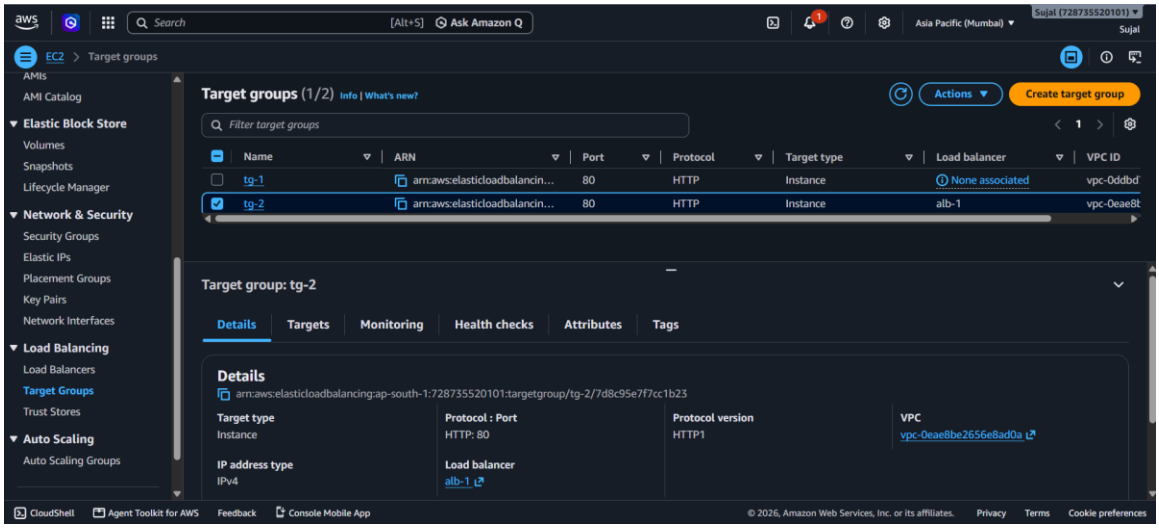
11. Step 8 - AMI Creation

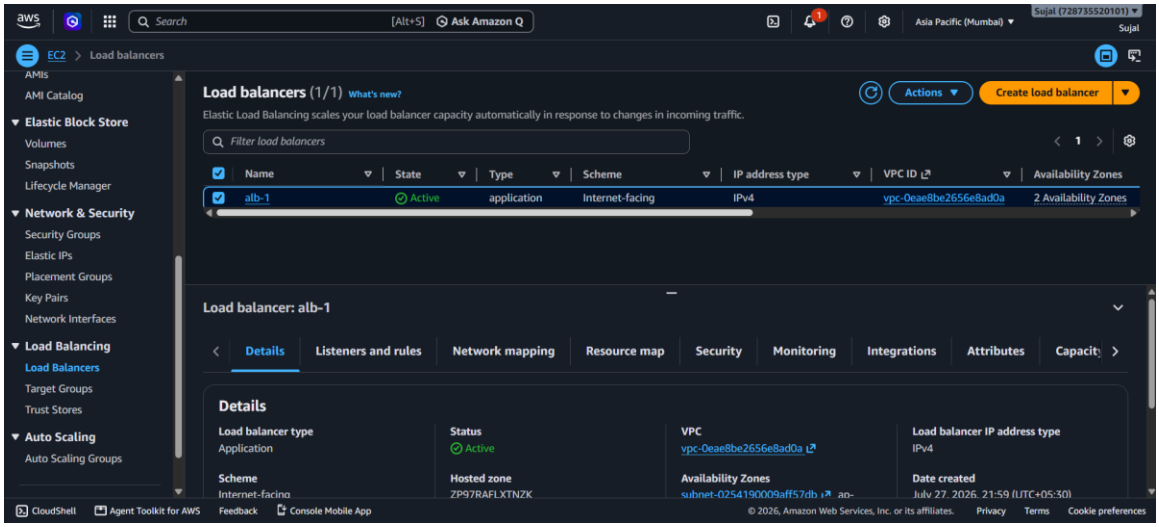
Created custom AMI (app-server-image) from the configured App Server to serve as the golden image for Auto Scaling.



12. Step 9 - Application Load Balancer & Target Group

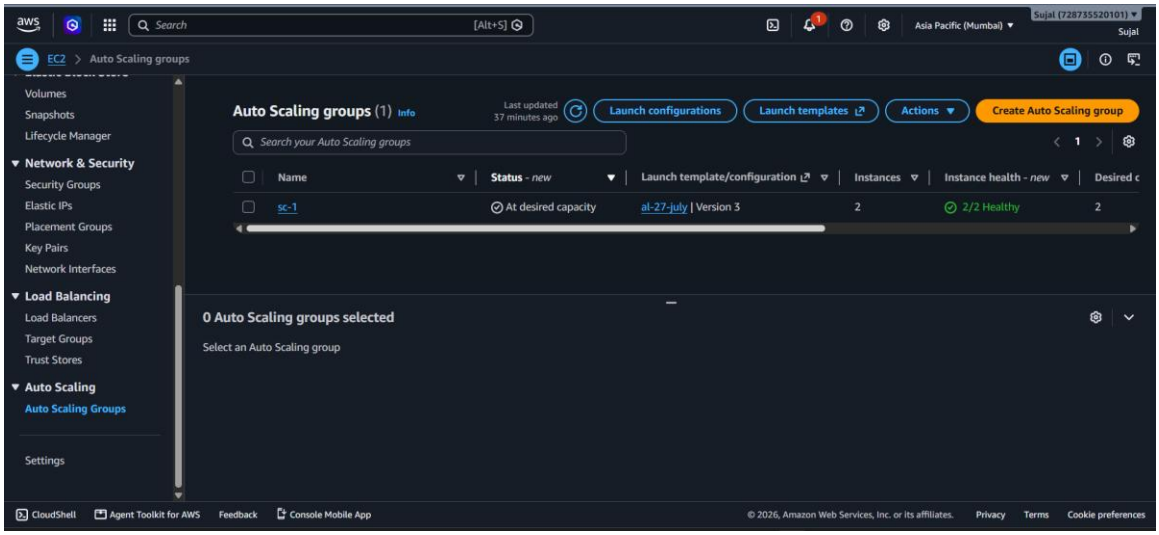
Created Target Group for HTTP Port 80. Provisioned Internet-facing Application Load Balancer across multi-AZ subnets forwarding to the Target Group.


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Sujal (728735520101)
Sujal
EC2 > Target groups
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Target groups (1/2) Info | What's new?
Actions
Create target group
Filter target groups
Name
ARN
Port
Protocol
Target type
Load balancer
VPC ID
tg-1
arn:aws:elasticloadbalancin...
80
HTTP
Instance
None associated
vpc-0ddbd
tg-2
arn:aws:elasticloadbalancin...
80
HTTP
Instance
alb-1
vpc-0eae8
Target group: tg-2
Details
Targets
Monitoring
Health checks
Attributes
Tags
Details
arn:aws:elasticloadbalancing:ap-south-1:728735520101:targetgroup/tg-2/7d8c95e7f7cc1b23
Target type
Instance
Protocol : Port
HTTP: 80
Protocol version
HTTP1
VPC
vpc-0eae8be2656e8ad0a
IP address type
IPv4
Load balancer
alb-1
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search
[Alt+S]
Ask Amazon Q
Asia Pacific (Mumbai)
Sujal (728735520101)
Sujal
EC2 > Load balancers
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
Target Groups
Trust Stores
Auto Scaling
Auto Scaling Groups
Load balancers (1/1) What's new?
Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.
Actions
Create load balancer
Filter load balancers
Name
State
Type
Scheme
IP address type
VPC ID
Availability Zones
alb-1
Active
application
Internet-facing
IPv4
vpc-0eae8be2656e8ad0a
2 Availability Zones
Load balancer: alb-1
Details
Listeners and rules
Network mapping
Resource map
Security
Monitoring
Integrations
Attributes
Capacity
Details
Load balancer type
Application
Status
Active
VPC
vpc-0eae8be2656e8ad0a
Load balancer IP address type
IPv4
Scheme
Internet-facing
Hosted zone
ZP97RAFLXTNZK
Availability Zones
Date created
July 27, 2026, 21:59 (UTC+05:30)
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

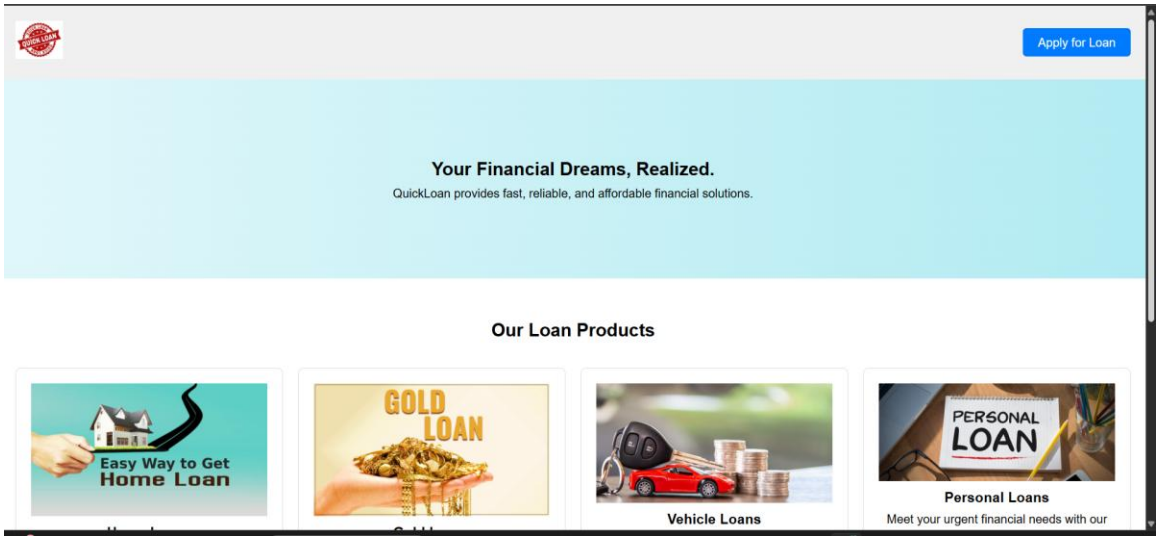
13. Step 10 - Auto Scaling Group Configuration

Configured Launch Template using the custom AMI. Created Auto Scaling Group (Min 2, Max 5 instances) attached to the ALB target group.



14. Step 11 - End-to-End Verification

Application homepage loaded correctly via domain. Submitted test loan application via web form and verified data entry directly in the RDS MySQL database.



15. Roles & Responsibilities

#	Responsibility	Status
1	Designed and created the VPC, subnets, route tables, IGW, and NAT Gateway.	Completed
2	Configured Security Groups to allow necessary traffic (80/443 for EC2, 3306 for RDS).	Completed
3	Deployed and tested PHP app on EC2 with Nginx and PHP-FPM; created custom AMI.	Completed
4	Configured S3 bucket for static asset hosting and updated application source files.	Completed
5	Provisioned Amazon RDS MySQL; created database schema and verified connectivity.	Completed
6	Configured Launch Templates and Auto Scaling Groups for horizontal scaling.	Completed
7	Implemented Application Load Balancer (ALB) with a Target Group for traffic distribution.	Completed
8	Performed end-to-end application testing and database verification.	Completed

16.Database Screenshots

```
MySQL [quickloan_db]> select*from applications;
+----+------------------------+-------------+---------------------------+-----------+
| id | name                   | contact     | email                     | loan_type |
+----+------------------------+-------------+---------------------------+-----------+
|  1 | sujal surendra phadale | 09421856717 | sujal.phadale23@pcu.edu.in | Home Loan |
+----+------------------------+-------------+---------------------------+-----------+
```